ACS-2947-002/003

Lab 1

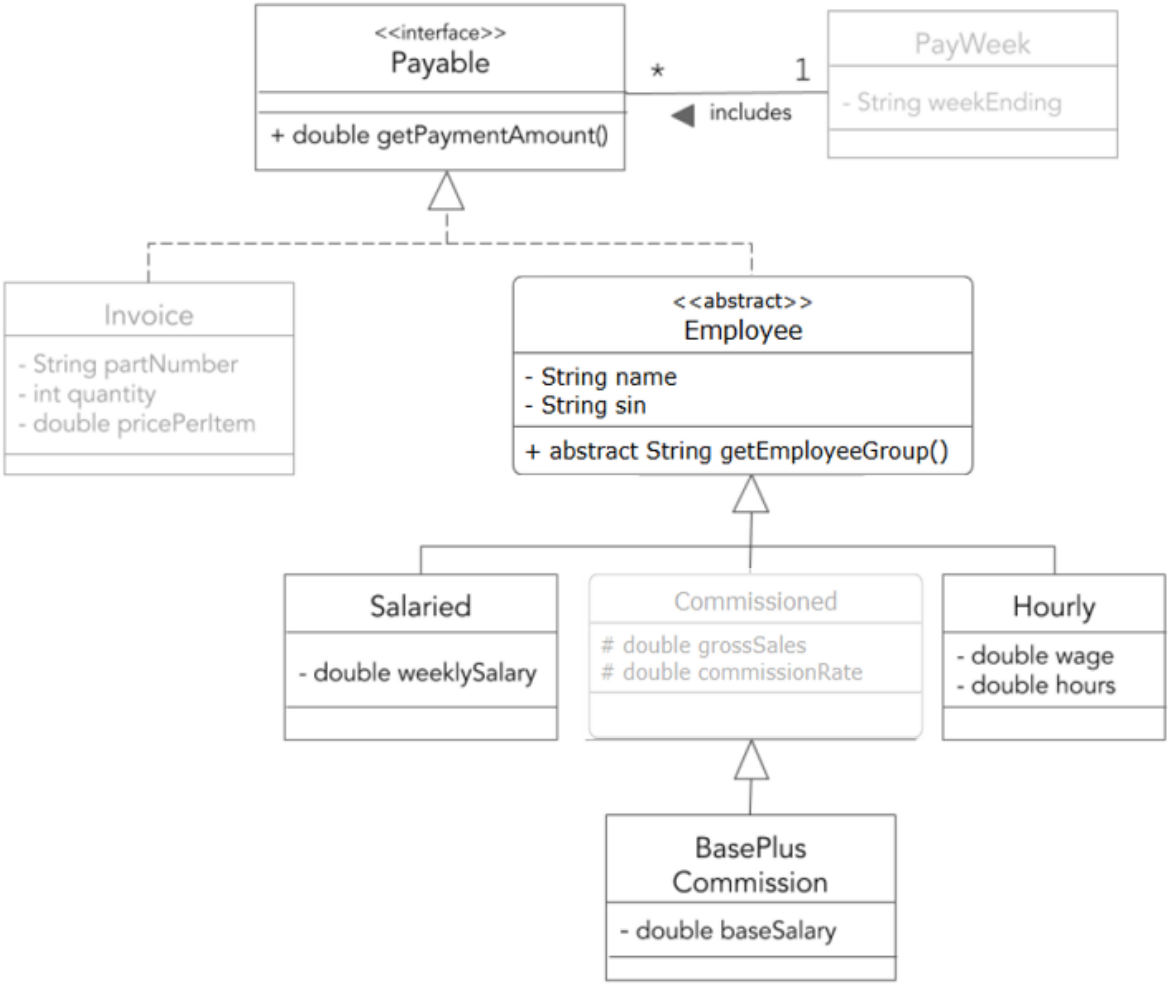
Due by Tuesday, January 14, 11:59 PM

Instructions

- The goal of this lab is to understand the interaction between different classes and interfaces, understand the use of *this* and *super,* understand the purpose of the *toString* method, and explore some of the Object-oriented programming paradigms we discussed in class.
- Submit your `.java` files (together in a `Lab1.zip` file) via Nexus.
- Include your name and student number as a comment in every file.

Task

Refer to the following class diagram that shows the different categories of entities that are *Payable* in a company (that is, invoices and employees) and the different categories of employees according to how they are paid.



The classes depicted in gray above are given in the provided `Lab1_files.zip`:

- Lab1_Driver
- Invoice
- Commissioned
- PayWeek

a) In the Employee hierarchy, create the classes for `Employee`, `Salaried`, `Hourly`, and `BasePlusCommission`.
b) Include a full-argument constructor and a no-argument constructor for ALL the classes.
c) Create the `Payable` interface. *Take some time to think about the applicable classes in which its method(s) should be implemented.*
d) Implement `getPaymentAmount` as follows for the following employee categories:

   Commissioned: *grossSales* * *commissionRate*
   BasePlusCommission: *baseSalary* + *grossSales* * *commissionRate*

   Implement `getPaymentAmount` in any other necessary classes. The calculation for each necessary class is based on the intuitive meaning of the variables in the respective classes.
e) Implement `getEmployeeGroup` in all necessary classes to return the type of employee group of an employee e.g., *Salaried Employee, Hourly Employee, Commissioned Employed or Base-plus-commission Employee*.
f) Include a `toString` method and any other necessary methods for the program in the appropriate class(es). T*he sample output below should be generated without modifying the provided driver class.*

Note: you are expected to use `this` and `super` keywords to enable code reuse as much as possible throughout this lab. *You will be penalized for not reusing code where possible.*

Note on `Lab1_Driver`

In this company, commissioned employees are paid weekly while salary/hourly employees get paid biweekly. The week ending January 17, 2025, is a week where **only commissioned** employees get paid. `Lab1_Driver` creates:

a) instances of all employee types (and adds them to an `ArrayList`)
b) two invoices
c) a `PayWeek` object for the week ending January 17 and then adds applicable employees and the given invoices.

Sample output:

```
Payments for the week ending January 17, 2025:
Invoice# 311: $799.92
Invoice# 312: $303.60
Employee# 1070: $1221.52 (Commissioned Employee)
Employee# 7770: $1449.35 (Base-plus-commission Employee)
```

Submission

Submit your **`Lab1.zip`** file that includes all the lab files (`Payable.java`, `Employee.java`, `Salaried.java`, `Hourly.java`, `BasePlusCommission.java`, `Lab1_Driver.java`, `PayWeek.java`, `Invoice.java`, `Commissioned.java`) via **Nexus**.